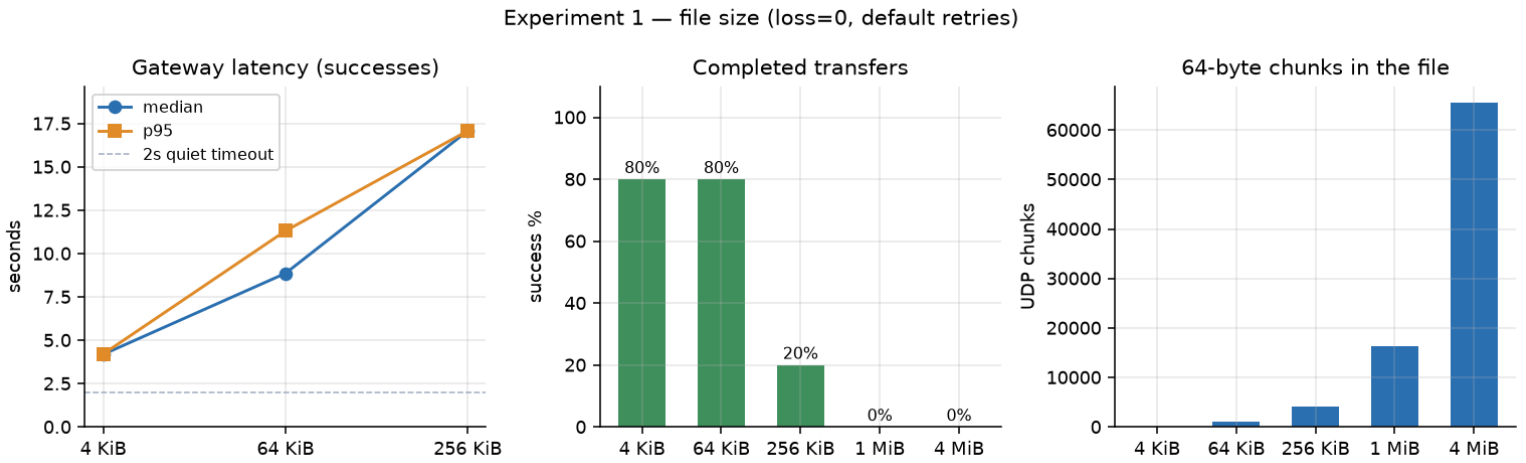
Three experiments on a lossy UDP file service

Client → ALB → gateway (HTTP, MAX_INFLIGHT=4) → NLB → backend (UDP, WORKERS=8) → S3. 64-byte chunks, 4 MiB cap.

	1. File size	2. Send loss	3. Load / scale
HW tool	HW4 sweep	HW4 sweep	HW5/6 Locust + watch
Fixed	SEND_LOSS=0, retries 5/5	64 KiB (1024 chunks)	64 KiB, 25 users, 8 min
Vary	4 KiB → 4 MiB	SEND_LOSS 0 then 10	concurrency; two scaler metrics

Gateway scales on ALBRequestCountPerTarget = 40. Backend scales on CPU = 40%. Min 2 / max 4 tasks each.

On the NLB path, size kills success, not just latency

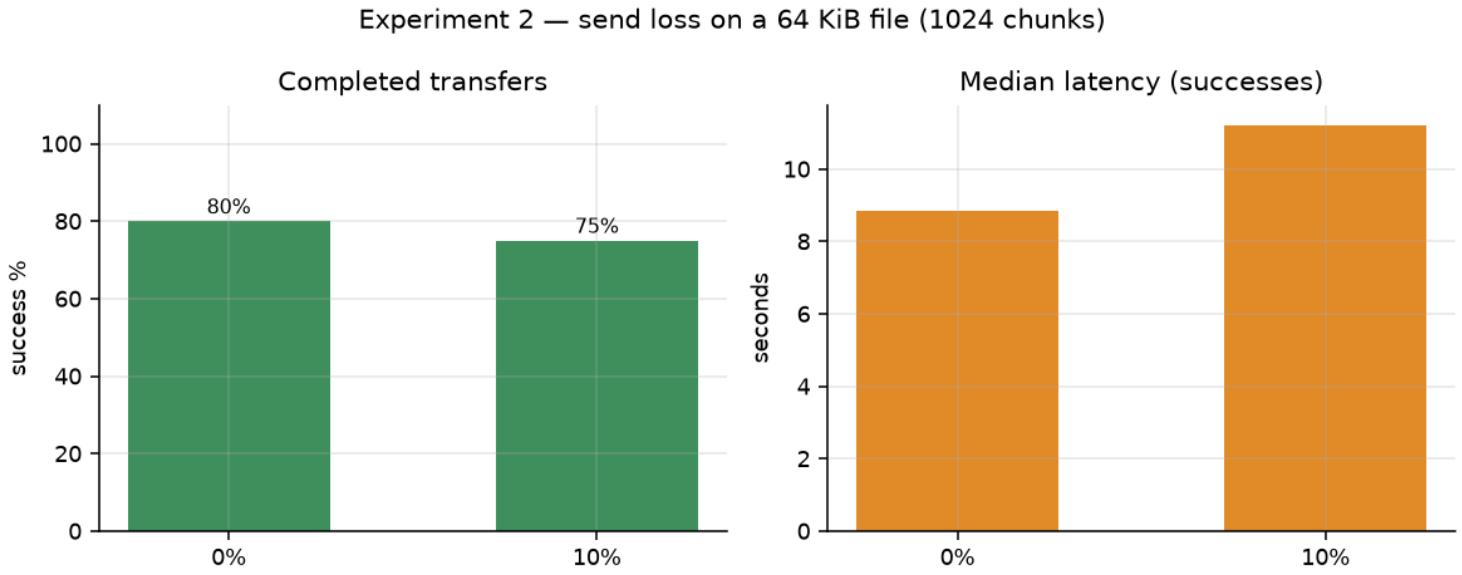


- 4 KiB (64 chunks): 4/5 ok, ~4 s
- 64 KiB (1024): 4/5 ok, ~9 s
- 256 KiB (4096): 1/5 ok
- 1 MiB and 4 MiB: 0/5 and 0/2
- Localhost: every size finished in 2.0–2.3 s. The 2 s quiet timeout is the floor.
- AWS blasts chunks through a UDP NLB. Tail-drops leave missing ranges; RETY bursts overflow the 32-deep worker queue.

HW4-style sweep. SEND_LOSS=0. Extra seconds are retry rounds, not send time.

EXPERIMENT 2 · SEND LOSS

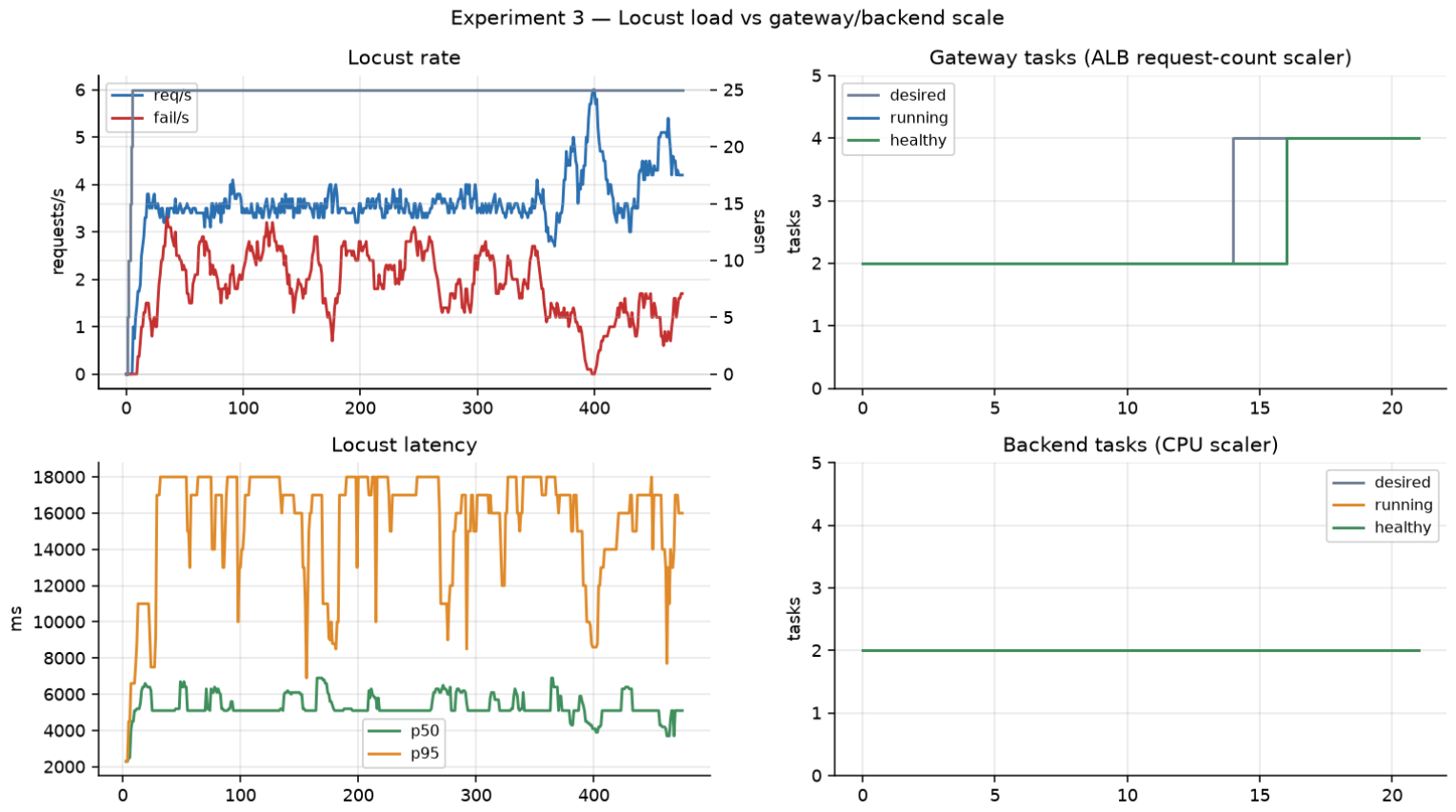
AWS is already lossy. +10% send loss makes 64 KiB slower, not much rarer.



- 64 KiB @ 0%: 80% ok, median 8.9 s
- 64 KiB @ 10%: 75% ok, median 11.2 s
- 4 KiB @ 10%: 8/8 ok, 4.3–6.4 s
- Locally, 10% finished every 64 KiB file (10–12 s) and 20% collapsed to 1/8 at exactly 12 s.
- On AWS, 1024-chunk files already sit near the 5-round budget. Small files still have slack.

Backend logs: Overloaded: dropped incoming datagram, plus consecutive missing-chunk ranges.

Gateway scaled 2 → 4. Backend never moved.



- 25 users, 8 min, 64 KiB: 1764 req, 51% fail
- 789 × 503 busy, 105 × 504 timeout
- Gateway desired 4 at 18:54, healthy 4 at 18:55
- Backend stayed at 2. CPU avg 22–39% (target 40). One task hit 76% max; target tracking uses the average.
- ALB can scale on requests/target. NLB cannot, so the backend only watches CPU.

Three limits, confirmed on AWS

1. Burst + 2 s completion

Localhost hides it — everything is ~2 s. On AWS, 256 KiB+ transfers mostly 504 because a blast of 64-byte datagrams is dropped and five RETY rounds cannot catch up.

2. Retry turns into overload

Missing ranges become RETY floods. The backend drops them once 32 jobs are queued. Extra SEND_LOSS mainly adds more 2 s quiet-timeout rounds.

3. The two scalers do not agree

The gateway feels request pile-up and scales. The backend, busy enough to 504 files, still averages under 40% CPU and stays at min=2.